

Dokumentacja użytkownika języka DAPL

Spis treści

1. Wprowadzenie
2. Instalacja
3. Uruchamianie programów
4. Składnia języka
5. Typy danych
6. Zmienne i stałe
7. Operatory
8. Instrukcje sterujące
9. Funkcje
10. Struktury danych
11. Operacje na tabelach
12. Biblioteki
13. Przykłady użycia
14. Obsługa błędów

1. Wprowadzenie

DAPL (Data Analysis Programming Language) to język programowania zaprojektowany specjalnie do analizy danych. Oferuje intuicyjną składnię dla operacji na danych oraz funkcjonalności analityczne.

1.1 Główne cechy

- **Prostota:** Czytelna składnia inspirowana popularnymi językami programowania
- **Analiza danych:** Wbudowane wsparcie dla struktur tabelarycznych
- **Bezpieczeństwo:** Statyczne typowanie z automatycznymi konwersjami
- **Funkcjonalność:** Pełne wsparcie dla programowania proceduralnego

2. Instalacja

2.1 Wymagania systemowe

- Python 3.8 lub nowszy
- Pakiet ANTLR4 dla Pythona

2.2 Kroki instalacji

1. Sklonuj repozytorium DAPL:

```
git clone https://github.com/twoje-repo/dapl.git
cd dapl
```

Utwórz środowisko wirtualne i zainstaluj zależności:

```
python -m venv venv

$ source venv\Scripts\activate #For Windows
$ source venv/bin/activate # For Linux

$ pip install -r requirements.txt
```

Ustaw zmienną środowiskową DAPL_Home

- Dla Windowsa

```
set DAPL_HOME=%cd%
```

- Dla Linuxa

```
export DAPL_HOME="$(pwd)"
```

Dodaj DAPL do path

- Dla Windowsa

```
set PATH=%PATH%;%DAPL_HOME%\bin
```

- Dla Linuxa

```
export PATH="$PATH:$DAPL_HOME/bin"
```

3. Uruchamianie programów

3.1 Podstawowe uruchomienie

```
./dapl [nazwa_pliku.dapl]
```

3.2 Przykład

```
# Uruchomienie programu hello.dapl
./dapl hello.dapl
```

3.3 Struktura pliku programu

Programy DAPL mają rozszerzenie `.dapl` i zawierają kod źródłowy w formacie tekstowym.

4. Składnia języka

4.1 Podstawowa struktura

```
// To jest komentarz jednoliniowy

/*
    To jest komentarz
    wieloliniowy
*/

// Każda instrukcja kończy się średnikiem
out("Hello, World!");
```

4.2 Konwencje nazewnictwa

- **Zmienne:** małe i wielkie litery, cyfra nie może być na początku (`myvariable`)
- **Funkcje:** małe i wielkie litery (`calculateSum`)
- **Stałe:** wielkie litery (`PI` , `MAXVALUE`)

5. Typy danych

5.1 Podstawowe typy

Typ	Opis	Przykład
<code>int</code>	Liczby całkowite	<code>42</code> , <code>-17</code> , <code>0</code>
<code>double</code>	Liczby zmiennoprzecinkowe	<code>3.14</code> , <code>-2.5</code> , <code>1.0</code>
<code>string</code>	Łańcuchy znaków	<code>"Hello"</code> , <code>"Tekst"</code>
<code>bool</code>	Wartości logiczne	<code>true</code> , <code>false</code>
<code>auto</code>	Automatyczne typowanie	Typ ustalany automatycznie

5.2 Automatyczne konwersje typów

```
double: d = 5;           // int → double (5.0)
string: s = 42;          // int → string ("42")
int: i = 3.14;           // double → int (3)
```

6. Zmienne i stałe

6.1 Deklaracja zmiennych

Zmienna z wartością domyślną:

```
var int: counter;       // counter = 0
var string: text;        // text = ""
var bool: flag;          // flag = false
```

Zmienna z inicjalizacją:

```
var int: age = 25;
var string: name = "Jan";
var bool: active = true;
```

6.2 Deklaracja stałych

```
int: PI = 3.14159;
string: GREETING = "Witaj!";
bool: DEBUGMODE = true;
```

6.3 Przypisywanie wartości

```
var int: x = 10;
x = 20;           // Zmiana wartości
x = x + 5;         // x = 25
```

7. Operatory

7.1 Operatory arytmetyczne

```
var int: a = 10;
var int: b = 3;

int: sum = a + b;      // 13
int: diff = a - b;     // 7
int: product = a * b;  // 30
int: quotient = a / b; // 3
int: remainder = a % b; // 1
double: quotient = a / b // 0.333333
double: whole_number = a /! b // 3.0
```

7.2 Operatory logiczne

```
var bool: x = true;
var bool: y = false;

bool: and_result = x and y; // false
bool: or_result = x or y;   // true
bool: not_result = !x;      // false
```

7.3 Operatory porównania

```
var int: a = 5;
var int: b = 10;

bool: equal = (a == b);      // false
bool: not_equal = (a != b);  // true
bool: less = (a < b);        // true
bool: greater = (a > b);     // false
bool: less_eq = (a <= b);    // true
bool: greater_eq = (a >= b); // false
```

7.4 Nawiasowanie

```
int: result = (2 + 3) * (4 - 1); // result = 15
bool: complex = (x or y) and (!z); // Złożone wyrażenie logiczne
```

8. Instrukcje sterujące

8.1 Instrukcja warunkowa if/elif/else

```
var int: score = 85;

if (score >= 90) {
    out("Ocena: A");
} elif (score >= 80) {
    out("Ocena: B");
} elif (score >= 70) {
    out("Ocena: C");
} else {
    out("Ocena: F");
}
```

8.2 Pętla while

```
var int: i = 1;
while (i <= 5) {
    out(i);
    i = i + 1;
}
// Wypisuje: 1, 2, 3, 4, 5
```

8.3 Pętla for

```
for (int: i = 1; i <= 10; i = i + 2) {
    out(i);
}
// Wypisuje: 1, 3, 5, 7, 9
```

8.4 Zagnieżdżanie instrukcji

```
for (int: i = 1; i <= 3; i = i + 1) {
    for (int: j = 1; j <= 3; j = j + 1) {
        out(i * j);
    }
}
```

9. Funkcje

9.1 Definicja funkcji

Funkcja zwracająca wartość:

```
function add(int: a, int: b): int {  
    return: a + b;  
}
```

Funkcja void (nie zwracająca wartości):

```
function greet(string: name): void {  
    out("Witaj");  
    out(name);  
}
```

Funkcja bez parametrów:

```
function getCurrentTime(): string {  
    return: "12:00:00";  
}
```

9.2 Wywołanie funkcji

```
// Wywołanie funkcji zwracającej wartość  
var int: sum = add(5, 3);      // sum = 8  
  
// Wywołanie funkcji void  
greet("Anna");                // Wypisuje: "Witaj,  
                             // Anna"  
  
// Wywołanie w wyrażeniu  
var int: result = add(10, add(2, 3)); // result = 15
```

9.3 Rekurencja

```
function factorial(int: n): int {  
    if (n <= 1) {  
        return: 1;  
    } else {  
        return: n * factorial(n - 1);  
    }  
}  
  
var int: fact5 = factorial(5); // fact5 = 120
```

9.4 Zasięgi zmiennych (scopes)

```
var int: global_var = 100;  
  
function test(): void {  
    var int: local_var = 50;  
    var int: global_var = 25; // Lokalna zmienna przysłania globalną  
  
    out(local_var);           // 50  
    out(global_var);          // 25 (lokalna)  
    out(^global_var);          // 100 (globalna, operator parent)  
}
```

9.5 Operator parent (^)

```
{
  var int: x = 10;
  {
    var int: x = 20;
    {
      var int: x = 30;
      out(x);      // 30 (najbliższa)
      out(^x);     // 20 (jeden poziom wyżej)
      out(^^x);    // 10 (dwa poziomy wyżej)
    }
  }
}
```

10. Struktury danych

10.1 Listy

```
// Pusta lista
List<int> numbers;

// Lista z inicjalizacją
List<string> names = ["Anna", "Jan", "Piotr"];
```

10.2 Tabele

Deklaracja pustej tabeli:

```
Table<int:id, string:name, int:age> users;
```

Tabela z danymi:

```
Table<int:id, string:name, double:salary> employees = [
  [1, "Jan Kowalski", 5000.0],
  [2, "Anna Nowak", 5500.0],
  [3, "Piotr Wiśniewski", 4800.0]
];
```

Wczytywanie z pliku CSV:

```
Table data = read_csv("data.csv");
```

11. Operacje na tabelach

11.1 Wyświetlanie tabeli

```
employees.show();
```

11.2 Filtrowanie

```
// Filtrowanie po kolumnie 2 (salary > 5000)
employees.filter(2, x => x > 5000.0).show();
```

11.3 Aplikowanie funkcji

```
// Zwiększenie pensji o 10% (kolumna 2)
employees.apply(2, x => x * 1.1).show();
```

11.4 Grupowanie z agregacją

```
// Grupowanie po kolumnie 1, agregacja kolumny 2 (średnia)
employees.groupBy(1, 2, average).show();
```

11.5 Dodawanie kolumn

```
employees.addColumn(<bool:active>).show();
```

11.6 Dodawanie wierszy

```
employees.addRow([4, "Marta Zielińska", 5200.0]).show();
```

11.7 Usuwanie kolumn

```
employees.drop(1).show(); // Usuwa kolumnę 1 (name)
```

11.8 Łączenie tabel (JOIN)

```
// JOIN po numerach kolumn
table1.join(table2, 0, 1).show();

// JOIN po nazwach kolumn
table1.join(table2, "id", "user_id").show();
```

11.9 Agregacje dostępne

- `max` - maksimum
- `min` - minimum
- `average` - średnia
- `sum` - suma
- `count` - liczność

12. Biblioteki

12.1 Import bibliotek

```
import str; // Importuje wszystkie funkcje stringowe
from str import length; // Importuje tylko funkcję length
```

12.2 Biblioteka str

Funkcja `length` - długość stringa:

```
import str;

var string: text = "Hello World";
var int: len = length(text); // len = 11
out(len);
```

Funkcja `startsWith` - sprawdzanie prefiksu:

```
import str;

var string: text = "Hello World";
var bool: starts = startsWith(text, "Hello"); // starts = true
out(starts);
```

13. Przykłady użycia

13.1 Program "Hello World"

```
// hello.dapl
out("Hello, World!");
```

13.3 Analiza danych sprzedażowych

```
// sales_analysis.dapl
import str;

// Wczytanie danych z CSV
Table sales = read_csv("sales.csv");

out("=== ANALIZA SPRZEDAŻY ===");

// Wyświetlenie surowych danych
out("Surowe dane:");
sales.show();

// Filtrowanie sprzedaży powyżej 1000
out("Sprzedaż powyżej 1000:");
sales.filter(2, x => x > 1000).show();

// Grupowanie po sprzedawcy z sumą sprzedaży
out("Suma sprzedaży według sprzedawcy:");
sales.groupBy(1, 2, sum).show();

// Średnia sprzedaż
out("Średnia sprzedaż:");
sales.groupBy(1, 2, average).show();
```

13.4 Funkcja rekurencyjna - Fibonacci

```
// fibonacci.dapl
function fibonacci(int: n): int {
    if (n <= 1) {
        return n;
    } else {
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
}

// Obliczenie pierwszych 10 liczb Fibonacciego
for (int: i = 0; i < 10; i = i + 1) {
    out(fibonacci(i));
}
```

13.5 Praca z zasięgami zmiennych


```
// scopes.dap1
var int: globalcounter = 0;

function incrementglobal( ): void {
    globalcounter = globalcounter + 1;
}

function demoscopes( ): void {
    var int: globalcounter = 100; // Lokalna zmienna

    out(globalcounter);          // 100
    out(^globalcounter);         // 0

    // Modyfikacja globalnej przez operator ^
    ^globalcounter = ^globalcounter + 10;
    out(^globalcounter); // 10
}

demoscopes( );
incrementglobal( );
out(globalcounter); // 11
```

14. Obsługa błędów

14.1 Rodzaje błędów

Błędy syntaktyczne:

```
// BŁĄD: Brakujący średnik
var int: x = 5

// BŁĄD: Nieprawidłowa składnia
if x == 5 { // Brak nawiasów
    out("test");
}
```

Błędy semantyczne:

```
// BŁĄD: Podwójna deklaracja
var int: x = 5;
var int: x = 10; // DoubleDeclarationError

// BŁĄD: Użycie niezadeklarowanej zmiennej
out(undefined_variable); // UnknownNameException

// BŁĄD: Niezgodność typów
var int: number = "text"; // WrongTypeException
```

14.2 Przykłady komunikatów błędów

Błąd składniowy z sugestią:

```
Did you mean "true" at line 5, column 10?
```

Błąd podwójnej deklaracji:

```
[DoubleDeclarationError] Double declaration of 'x'
First declared at line 3
Second declaration: line 5, column 8
5: var int: x = 10;
    ^
```

Błąd nieznannej zmiennej:

```
[UnknownNameException] Use of undeclared name 'y' at line 7, column 4
7: out(y);
    ^
```

Błąd typu:

```
[WrongTypeException] Wrong type for 'age': expected 'int', but got 'string'
```

14.4 Debugowanie

Aby ułatwić debugowanie:

- Używaj komunikatów `out()` do śledzenia wykonania
- Sprawdzaj wartości zmiennych w kluczowych punktach
- Dziel kod na mniejsze funkcje
- Testuj każdą funkcjonalność osobno

14.5 Ograniczenia

- **Rekurencja:** Maksymalnie 30 poziomów zagnieżdżenia
- **Typy:** Brak typów użytkownika (tylko podstawowe)
- **Tablice:** Indeksowanie od 0
- **Pliki:** Obsługa tylko CSV w formacie UTF-8

Przykładowy plik CSV (test.csv)

```
id,name,age,salary
1,Jan Kowalski,30,5000
2,Anna Nowak,25,5500
3,Piotr Wiśniewski,35,4800
```